Laboratório Funcional · Aula 12

Pedro Massaki Carnelossi · RA 24036481-2 · Turma ESOFT6S-B
Gerado em 16/10/2026, 20:14:34 · tempo registrado no cronômetro: 18:15

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora(): o resultado depende do relógio, que é algo de fora da função e muda a cada instante. Duas chamadas iguais devolvem valores diferentes.
poe(lista, x): tem efeito colateral, porque o append altera a lista que foi passada como parâmetro.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2

4: 14

7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

O resultado não depende só do parâmetro. Ele depende da variável global contador, que a própria função altera a cada chamada. Com isso, a mesma entrada dá saídas diferentes. Isso quebra a transparência referencial: para testar a função, é preciso conhecer e controlar o estado global e a ordem das chamadas, então ela não pode ser testada sozinha

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(3·4) = f(12) = 12 + 2 = 14
compor(g, f)(4) = g(f(4)) = g(4 + 2) = g(6) = 3·6 = 18

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) devolve uma lista nova e deixa notas como estava (linha 7: [7, 3, 9]). Já notas.sort() ordena a própria lista, o que é um efeito colateral (linha 8). O estilo funcional é o sorted, porque não modifica dados e só devolve um valor.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (QUOTE) diz ao Scheme para não avaliar a lista e tratá-la como dado. Sem ele, o Scheme tentou avaliar (A B C) como uma chamada de função: A seria a função e B e C os parâmetros. Como A não está definida, deu erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme vai funcionar e imprimir 1000000, porque a recursão é em cauda e o Scheme faz a otimização da chamada em cauda, então a pilha não cresce. No Python, conta_ate(500, 0) imprime 500, mas a chamada com um milhão para com RecursionError. O Python não faz essa otimização e limita a pilha a cerca de 1000 chamadas.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
...
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, a pilha de registros de ativação cresce: há uma chamada pendente para cada n (4 com n = 4, 6 com n = 6), e cada uma espera a volta para fazer a multiplicação (* n ...). Na recursão em cauda, a pilha não cresce. A chamada recursiva é a última operação e o resultado parcial vai no acumulador, então nada fica esperando e o mesmo registro pode ser reaproveitado.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme é obrigado a fazer a otimização da chamada em cauda (p. 656). Ele reaproveita o registro de ativação, e o milhão de chamadas ocupa espaço constante, como um laço. O Python não faz essa otimização e ainda limita a profundidade da pilha a cerca de 1000 chamadas. Por isso ele empilhou registros até estourar o limite e lançou RecursionError: maximum recursion depth exceeded.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
```

```
       (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi pares, porque ela precisa construir uma lista nova com cons e decidir entre incluir ou pular o car.

null?: nas três funções, é o caso base (devolve 0, 0 ou '()).
car: tamanho não usa o valor (só soma 1); soma-quadrados eleva o car ao quadrado; pares testa se o car é par com even?.
cdr: as três chamam a si mesmas com o cdr, ou seja, com o resto da lista.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares → [0,1,4,9,16]. Por causa da avaliação preguiçosa, só os 5 primeiros elementos são calculados. primeiro 42 (div 1 0) → 42. O segundo parâmetro é ignorado e nunca é avaliado, então não há erro de divisão por zero.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa: os elementos de squares só são calculados quando alguém precisa deles. take 5 pede apenas os 5 primeiros, então o resto da lista nunca é gerado. Em primeiro 42 (div 1 0), a função ignora o segundo parâmetro (_). Como o Haskell não é estrito, div 1 0 nunca é avaliado e não há divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares encontra o 16 na 5ª posição e para. O 17 não é um quadrado perfeito, e member não tem como saber que deve parar: o caso [] nunca acontece numa lista infinita, então ele continua comparando para sempre.

```
member2 n (m:x)
 | m < n    = member2 n x
 | m == n   = True
 | otherwise = False
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) → 15

[1, 2, 3].map(vezes(5)) → [5, 10, 15]

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida é um fechamento (closure). Ela guarda uma ligação com o ambiente onde foi criada, e nesse ambiente n = 10. Esse valor capturado continua existindo enquanto mais10 existir, mesmo depois que somador já terminou

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g) aplica primeiro g e depois f: f(g(4)) = f(12) = 14
f.andThen(g) aplica primeiro f e depois g: g(f(4)) = g(6) = 18
O que corresponde ao compor(f, g) da Prática 1 é o f.compose(g).

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
```

```
  { nome: "Davi", nota: 9.5 },
];
const aprovados = alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
[50.0, 108.0, 80.0, 180.0]
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler. O pipeline diz o quê fazer (filtrar, transformar, somar) sem índices, sem acumulador e sem estado mutável. A versão com laço é mais fácil de depurar, porque dá para colocar um print ou um breakpoint dentro dele e ver cada valor intermediário. No pipeline isso exige recursos extras, como peek() em Java ou quebrar a cadeia em variáveis.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender: lê-se direto como “a soma dos cubos de 1 a n”, sem variável de controle, sem acumulador e sem condição de parada. Eu aceitaria o custo de cerca de 2 vezes no tempo de execução em sistemas onde clareza, correção e facilidade de teste valem mais que desempenho bruto: sistemas de negócio, processamento de dados, compiladores, backends web. Não aceitaria em sistemas com restrição forte de tempo ou de recursos: tempo real, sistemas embarcados, jogos, drivers, computação de alto desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o Haskell trabalhar com uma lista infinita e nunca avaliar o div 1 0, só porque o valor não era necessário.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

```
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]
```